Implementing a Secure Framework for a Code Review Tool

Project By

Siddharth Gaikwad

Title Explanation

A **Secure Code Review Framework** is a platform that automatically analyzes source code to identify security vulnerabilities, coding errors, insecure practices, and compliance violations before software is deployed. The framework combines multiple open-source security tools, coding standards, and automation features into a single system that helps developers write secure and high-quality code.

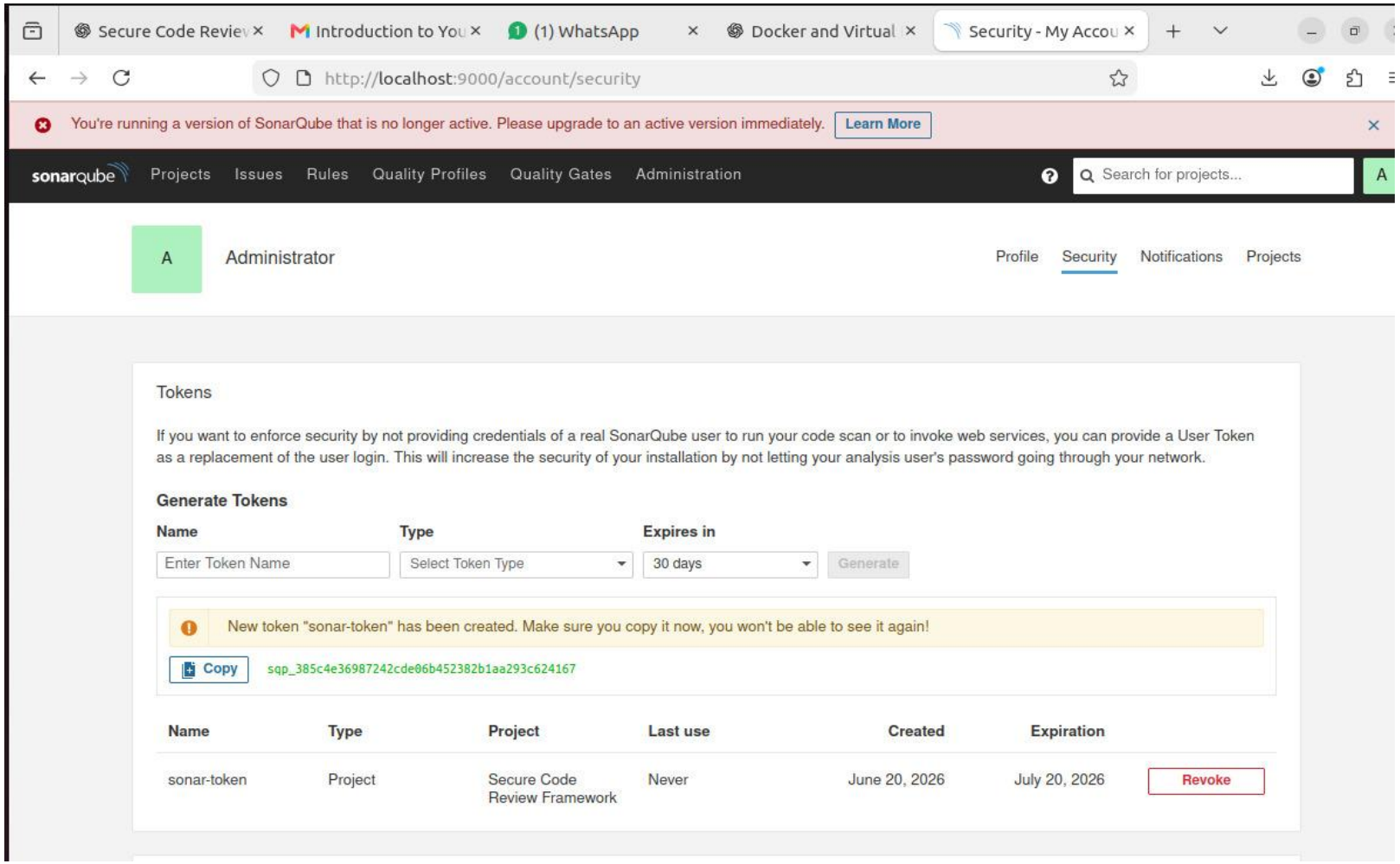
- The goal of this project is to create a centralized code review environment that can:
- Detect security vulnerabilities early in development.
- Enforce secure coding standards.
- Automate code scanning during development and deployment.
- Provide remediation guidance to developers.
- Track security compliance and code quality metrics.
- Support collaboration among developers, reviewers, and administrators

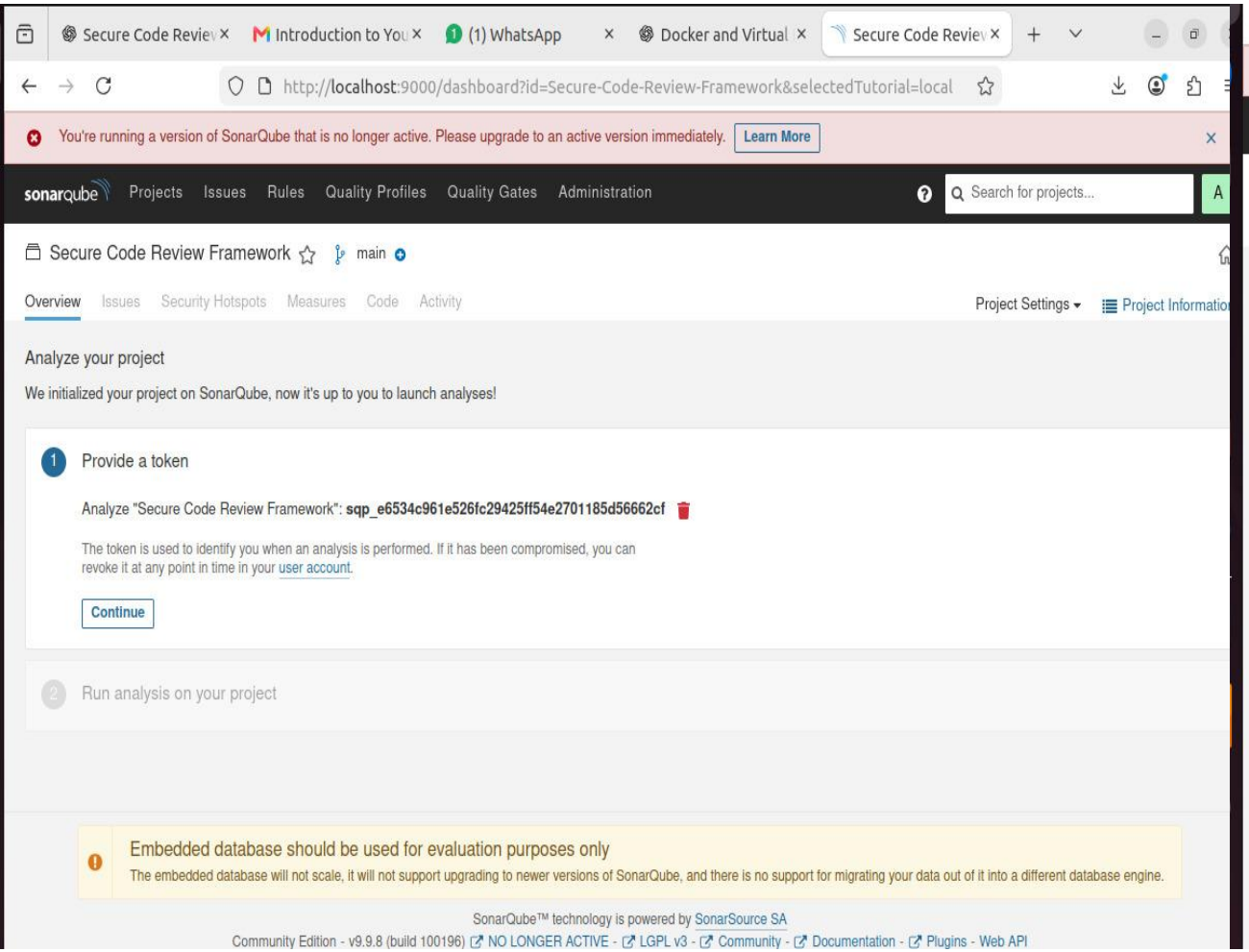
1. Introduction

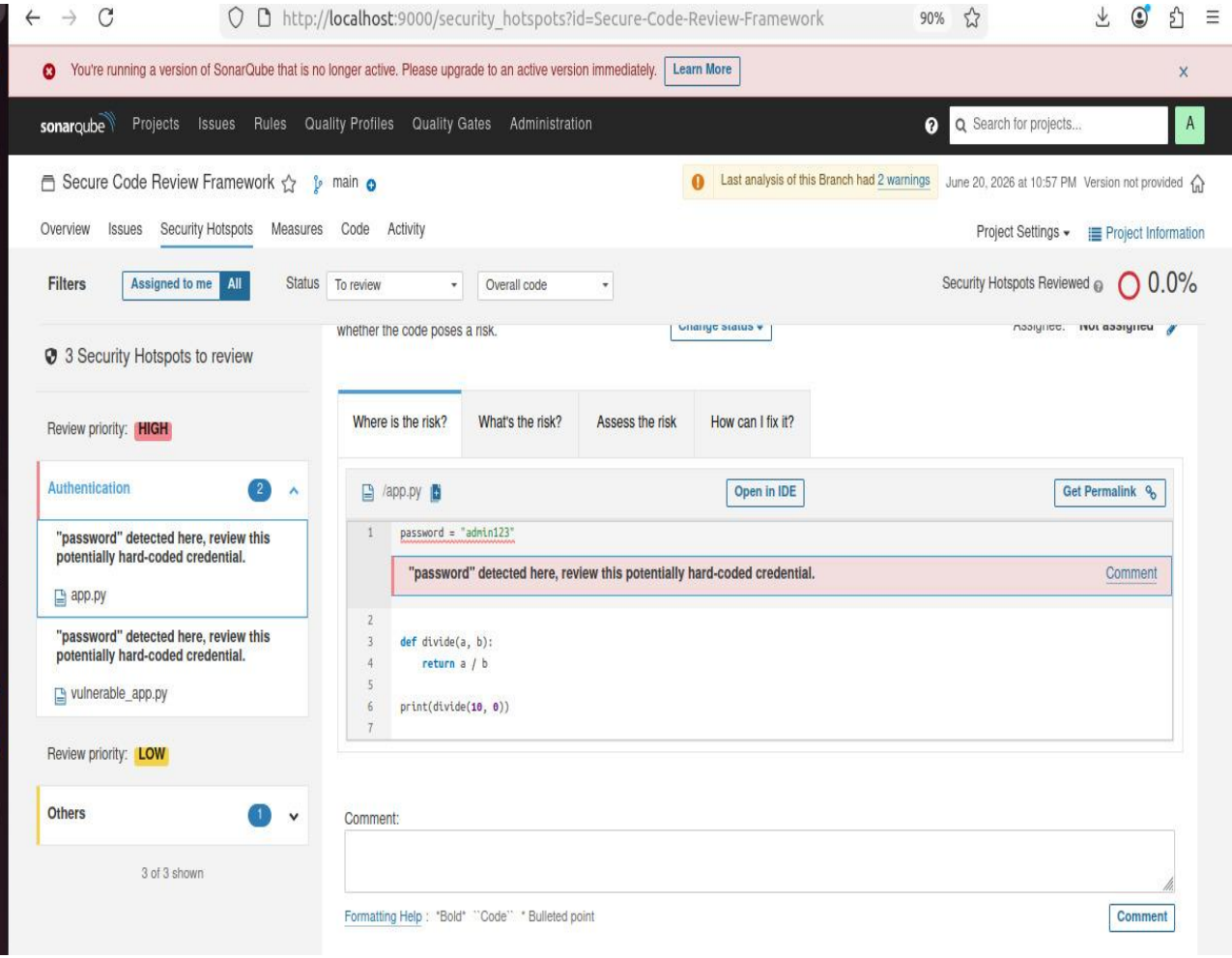
Modern software applications face increasing cybersecurity threats such as:

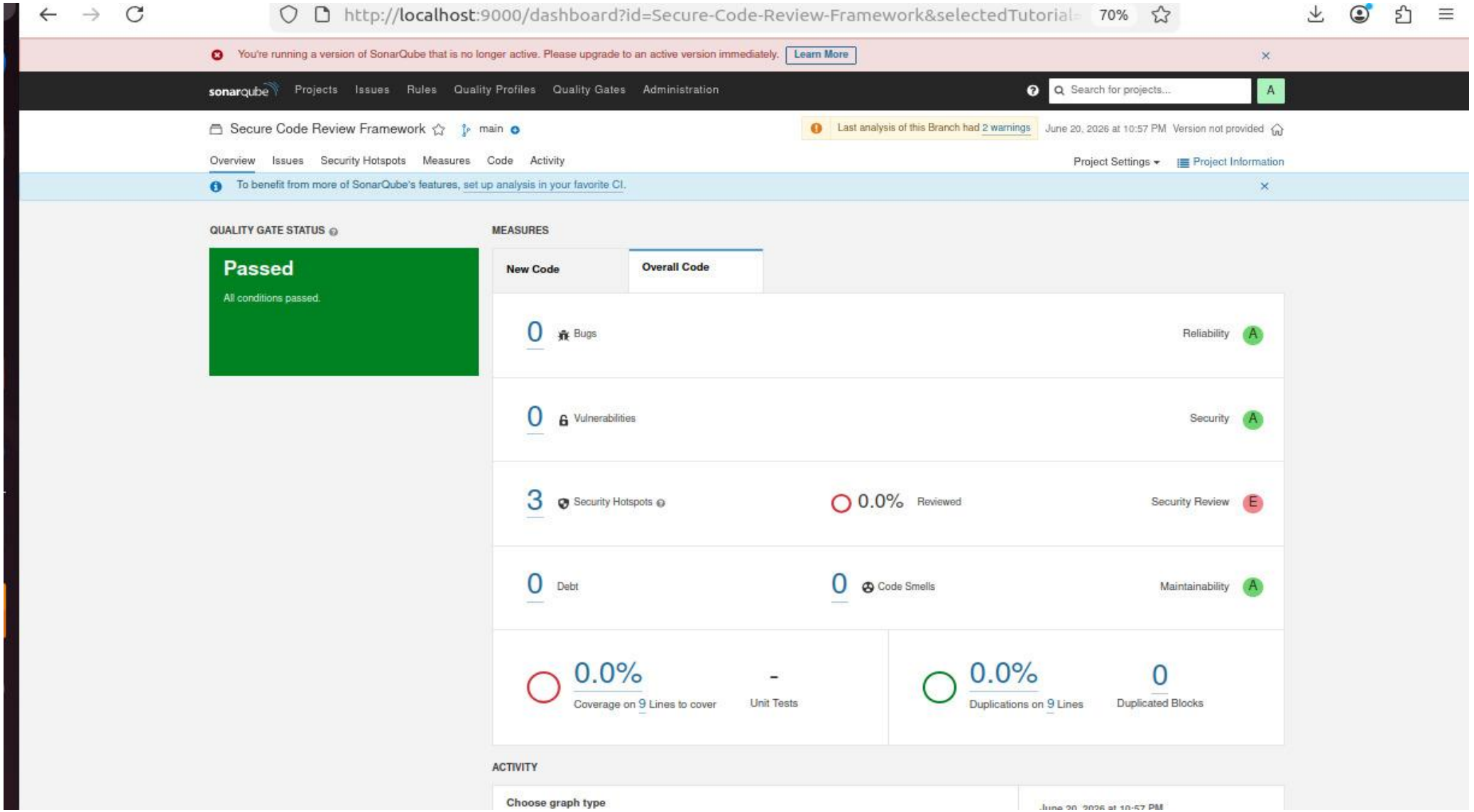
- SQL Injection
- Cross-Site Scripting (XSS)
- Authentication Bypass
- Hardcoded Secrets
- Vulnerable Third-Party Libraries
- Remote Code Execution
- Manual code reviews are often time-consuming and may miss critical vulnerabilities.
- A secure code review framework automates the review process using static analysis, vulnerability scanning, and security best practices to improve software security and development efficiency.

1. Framework Selection & Architecture Design




Secure Code Review
Introduction to You
(1) WhatsApp
Docker and Virtual
Secure Code Review
http://localhost:9000/dashboard?id=Secure-Code-Review-Framework&selectedTutorial=local
You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately.
Learn More
sonarqube
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
Search for projects...
Secure Code Review Framework
main
Overview
Issues
Security Hotspots
Measures
Code
Activity
Project Settings
Project Information
Analyze your project
We initialized your project on SonarQube, now it's up to you to launch analyses!
1
Provide a token
Analyze "Secure Code Review Framework": sqp_e6534c961e526fc29425ff54e2701185d56662cf
The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point in time in your user account.
Continue
2
Run analysis on your project
Embedded database should be used for evaluation purposes only
The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.
SonarQube™ technology is powered by SonarSource SA
Community Edition - v9.9.8 (build 100196) NO LONGER ACTIVE - LGPL v3 - Community - Documentation - Plugins - Web API


http://localhost:9000/security_hotspots?id=Secure-Code-Review-Framework
90%
You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately.
Learn More
sonarqube
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
Search for projects...
Secure Code Review Framework
main
Last analysis of this Branch had 2 warnings
June 20, 2026 at 10:57 PM
Version not provided
Overview
Issues
Security Hotspots
Measures
Code
Activity
Project Settings
Project Information
Filters
Assigned to me
All
Status
To review
Overall code
Security Hotspots Reviewed
0.0%
3 Security Hotspots to review
whether the code poses a risk.
Assignee: Not assigned
Review priority: HIGH
Authentication
2
"password" detected here, review this potentially hard-coded credential.
app.py
"password" detected here, review this potentially hard-coded credential.
vulnerable_app.py
Review priority: LOW
Others
1
3 of 3 shown
Where is the risk?
What's the risk?
Assess the risk
How can I fix it?
/app.py
Open in IDE
Get Permalink
1 password = "admin123"
"password" detected here, review this potentially hard-coded credential.
Comment
2
3 def divide(a, b):
4 return a / b
5
6 print(divide(10, 0))
7
Comment:
Formatting Help : *Bold* ``Code`` * Bulleted point
Comment


http://localhost:9000/dashboard?id=Secure-Code-Review-Framework&selectedTutorial=
70%
You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately.
Learn More
sonarqube
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
Search for projects...
A
Secure Code Review Framework
main
Last analysis of this Branch had 2 warnings
June 20, 2026 at 10:57 PM Version not provided
Overview
Issues
Security Hotspots
Measures
Code
Activity
Project Settings
Project Information
To benefit from more of SonarQube's features, set up analysis in your favorite CI.
QUALITY GATE STATUS
Passed
All conditions passed.
MEASURES
New Code
Overall Code
0 Bugs
Reliability A
0 Vulnerabilities
Security A
3 Security Hotspots
0.0% Reviewed
Security Review E
0 Debt
0 Code Smells
Maintainability A
0.0%
Coverage on 9 Lines to cover
-
Unit Tests
0.0%
Duplications on 9 Lines
0
Duplicated Blocks
ACTIVITY
Choose graph type

2.Security Analysis and Vulnerability Detection Results

- SonarQube successfully detected hardcoded credentials.
- Semgrep identified weak cryptography practices and exposed secrets.
- Gitleaks detected leaked API tokens and sensitive information.
- OWASP Dependency-Check installation and Java configuration were completed successfully.

```
siddhu@siddhu-virtual-machine:~/Secure-Code-Review-Framework/samp
le-project$ cd ~/Secure-Code-Review-Framework/sample-project

gitleaks detect --verbose



    gitleaks

Finding:     sonar.token=sqp_e6534c961e526fc29425ff54e2701185d566
62cf
Secret:      sqp_e6534c961e526fc29425ff54e2701185d56662cf
RuleID:      sonar-api-token
Entropy:     3.831051
File:        sonar-project.properties
Line:        5
Commit:      bee048de5f4eb36aa93a49cbabc01e72168609a2
Author:      Siddhu
Email:       siddhu@example.com
Date:        2026-06-20T20:29:23Z
Fingerprint: bee048de5f4eb36aa93a49cbabc01e72168609a2:sonar-proje
ct.properties:sonar-api-token:5

2:00AM INF 1 commits scanned.
2:00AM INF scanned ~466 bytes (466 bytes) in 140ms
2:00AM WRN leaks found: 1
siddhu@siddhu-virtual-machine:~/Secure-Code-Review-Framework/samp
le-project$
```

```
siddhu@siddhu-virtual-machine:~/Secure-Code-Review-Framework/samp
le-project$ gitleaks detect --report-format json --report-path gi
tleaks-report.json

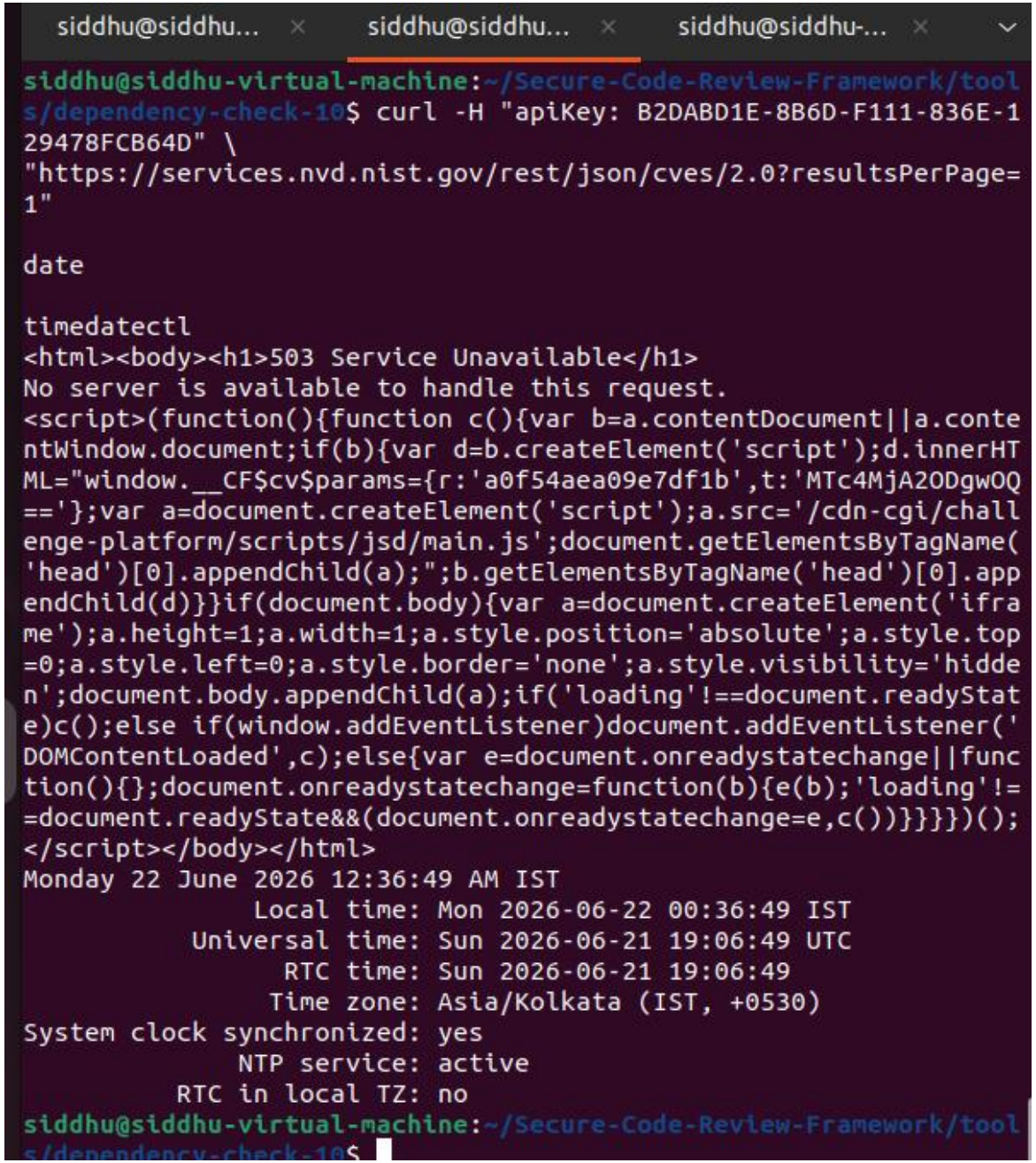


    gitleaks

2:01AM INF 1 commits scanned.
2:01AM INF scanned ~466 bytes (466 bytes) in 167ms
2:01AM WRN leaks found: 1
siddhu@siddhu-virtual-machine:~/Secure-Code-Review-Framework/samp
le-project$ ls
app.py                  sonar-project.properties
gitleaks-report.json    vulnerable_app.py
siddhu@siddhu-virtual-machine:~/Secure-Code-Review-Framework/samp
le-project$
```

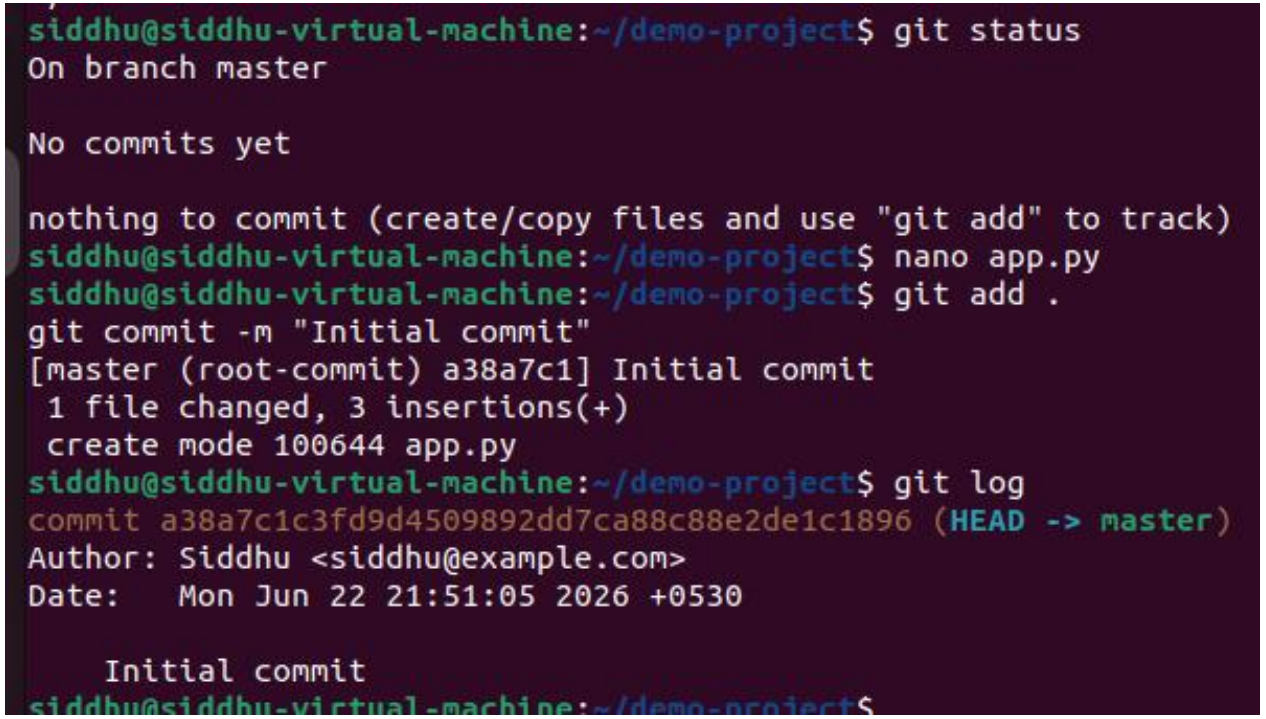
dependency scanning

- This is a common problem with newer Dependency-Check versions and NVD API updates, so it does not indicate a problem with your framework setup itself.
- The next step is to inspect the detailed log (odc. odc.1og) to determine whether this is an API response problem, a corrupter cache, or a specific Dependency-Check bug.

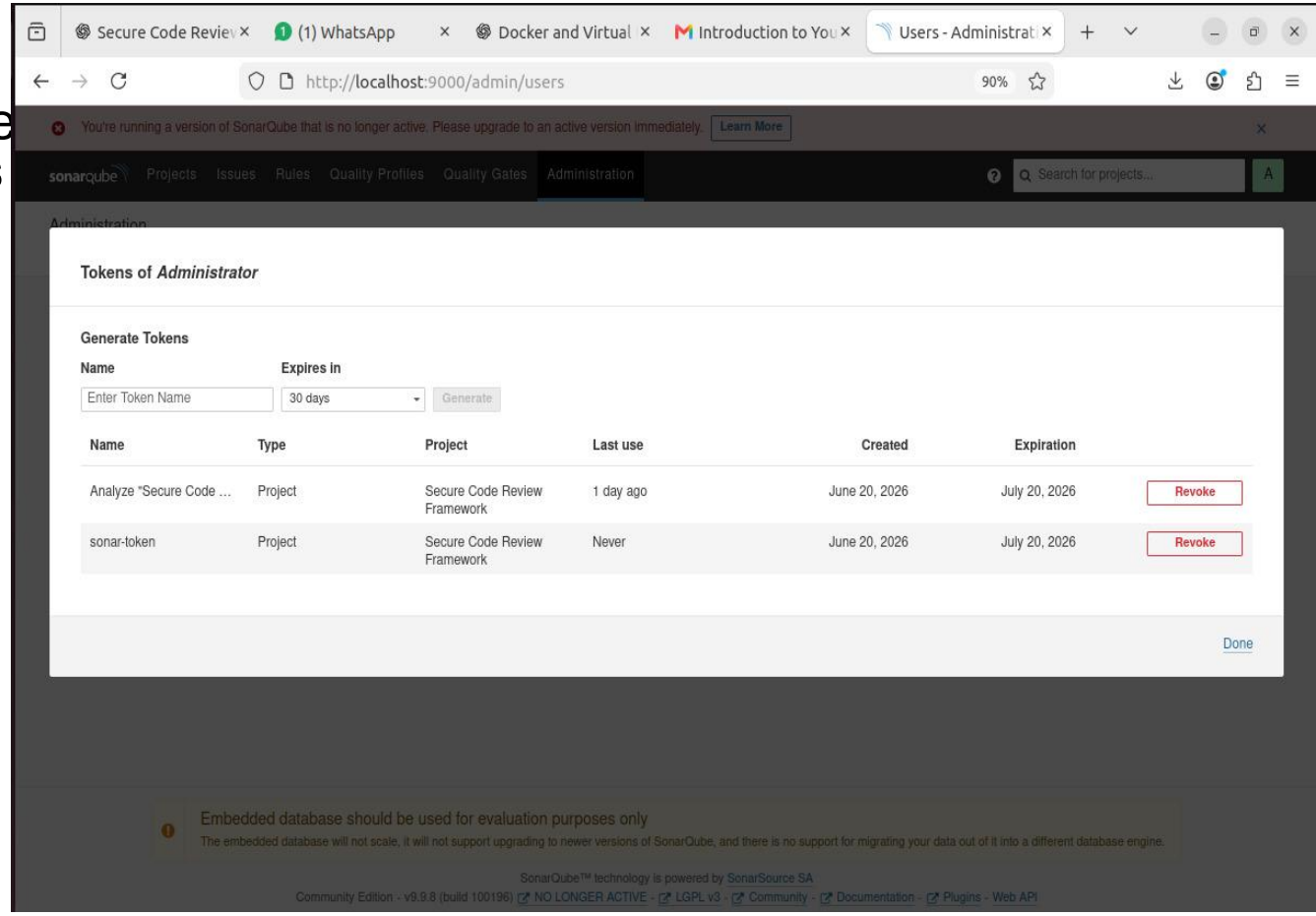


4. Version Control Integration

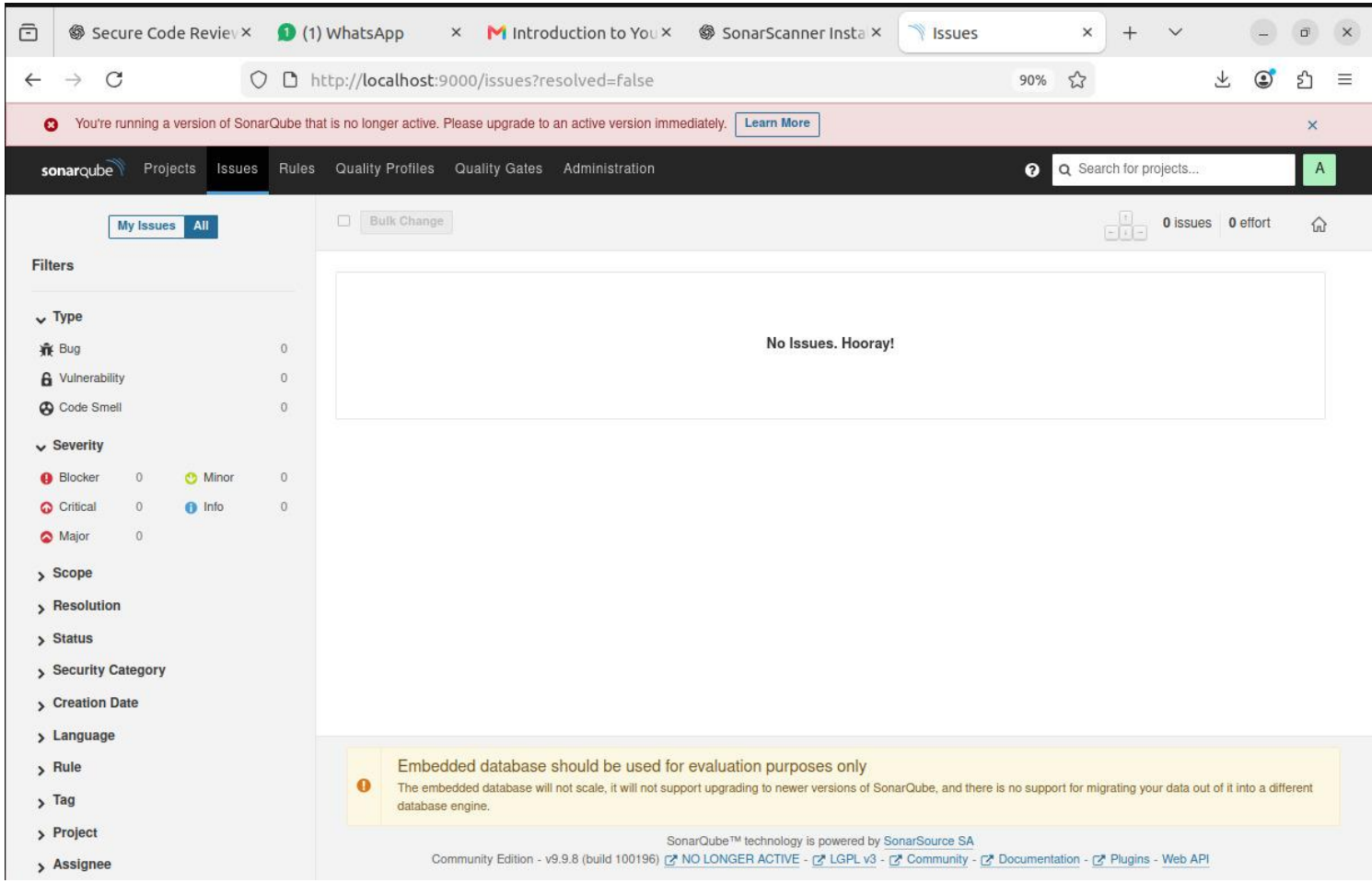
- **Introduction**
- Version Control Systems (VCS) are essential components of modern software development. They help developers track changes, collaborate efficiently, and maintain different versions of source code. In this project, Git was integrated with the Secure Code Review Framework to enable code tracking and automated security analysis.



- **Git Repository Configuration**
- A local Git repository was initialized to manage the project source code. The repository allows developers to maintain version history, track modifications, and revert changes when necessary.
- The following Git operations were performed:
- Repository initialization using git init
- Adding files using git add
- Creating commits using git commit
- Viewing commit history using git log
- The successful creation of the initial commit confirms that version control integration was properly established.
- **Figure:** Git repository initialization and initial commit.



- **Commit History Management**
- Git maintains a complete history of all project modifications. Each commit contains a unique commit ID, author information, date, and commit message. This enables developers to identify changes and track project evolution.
- The generated commit:
- Commit Message: **Initial commit**
- Commit ID: **a38a7c1**
- This commit serves as the baseline version of the project.

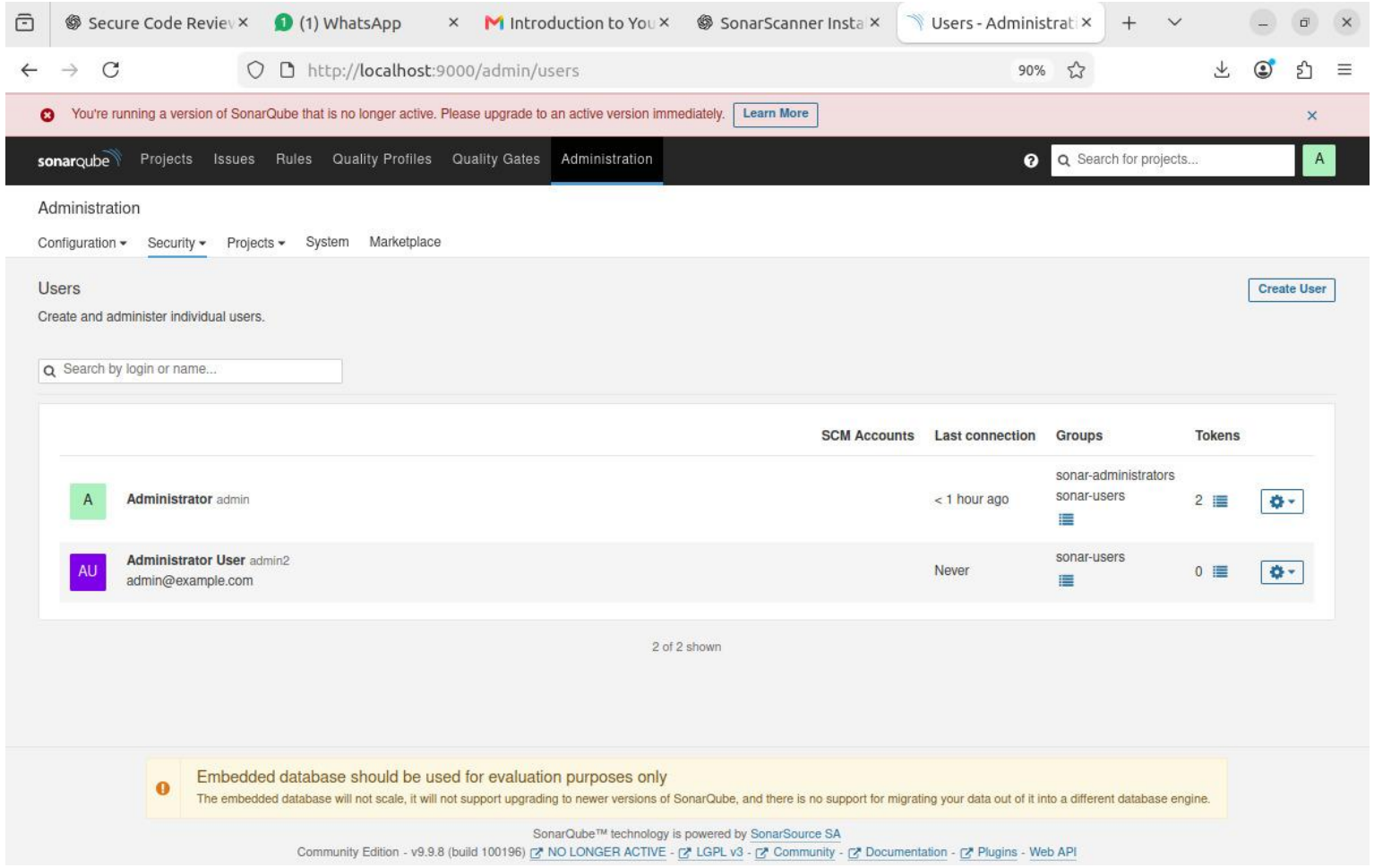


- **Conclusion**
- The successful integration of Git and automated security workflows demonstrates that the Secure Code Review Framework can effectively support version control, continuous monitoring, and secure software development practices. The implementation enables early vulnerability detection and improves overall code quality before deployment.

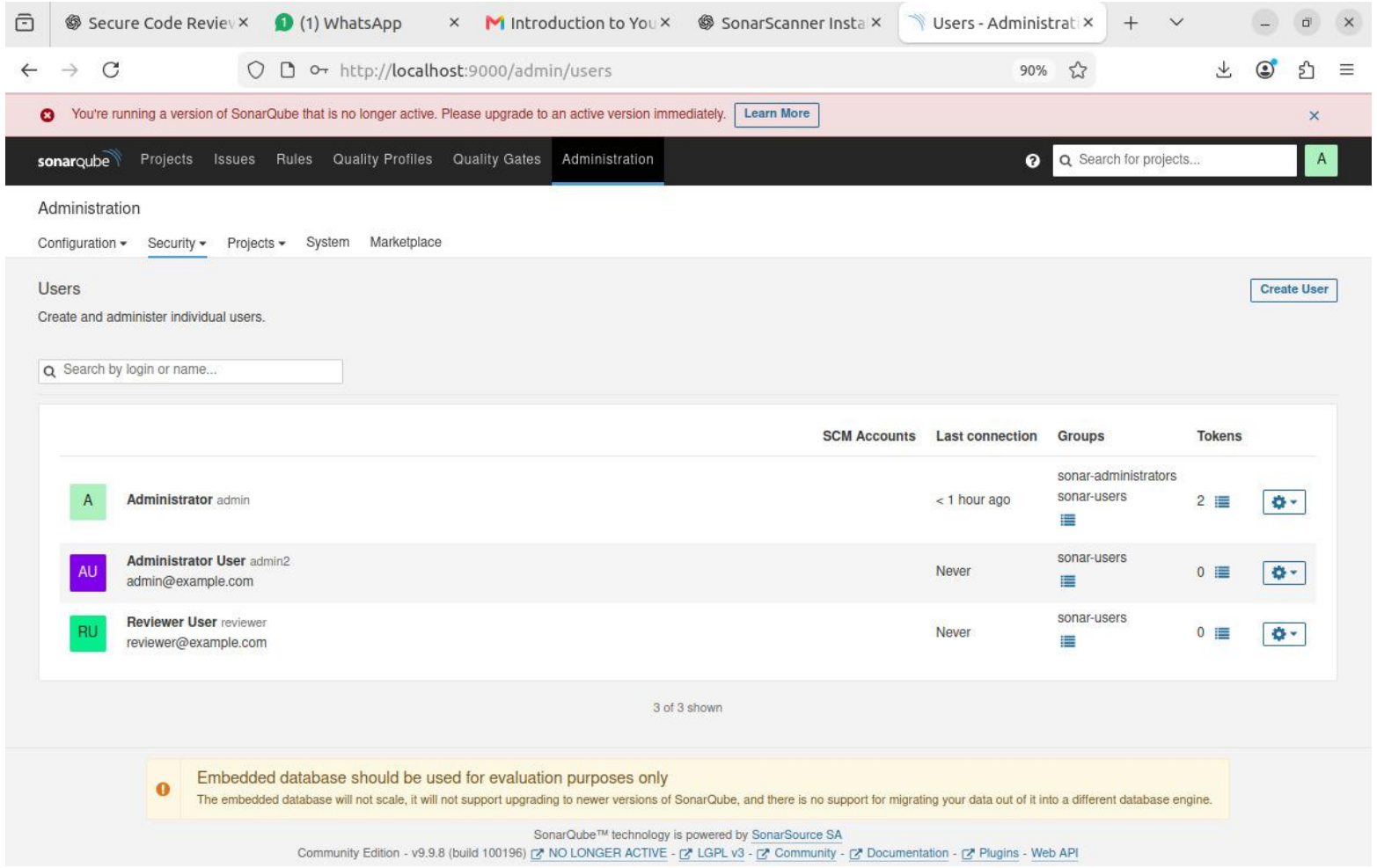
5.Authentication & Authorization

- **Introduction**
- Authentication and Authorization are essential security mechanisms in a secure code review framework. Authentication verifies the identity of users, while Authorization determines the permissions and access levels assigned to each user. Implementing these mechanisms prevents unauthorized access and ensures that users can perform only the actions permitted by their assigned roles.

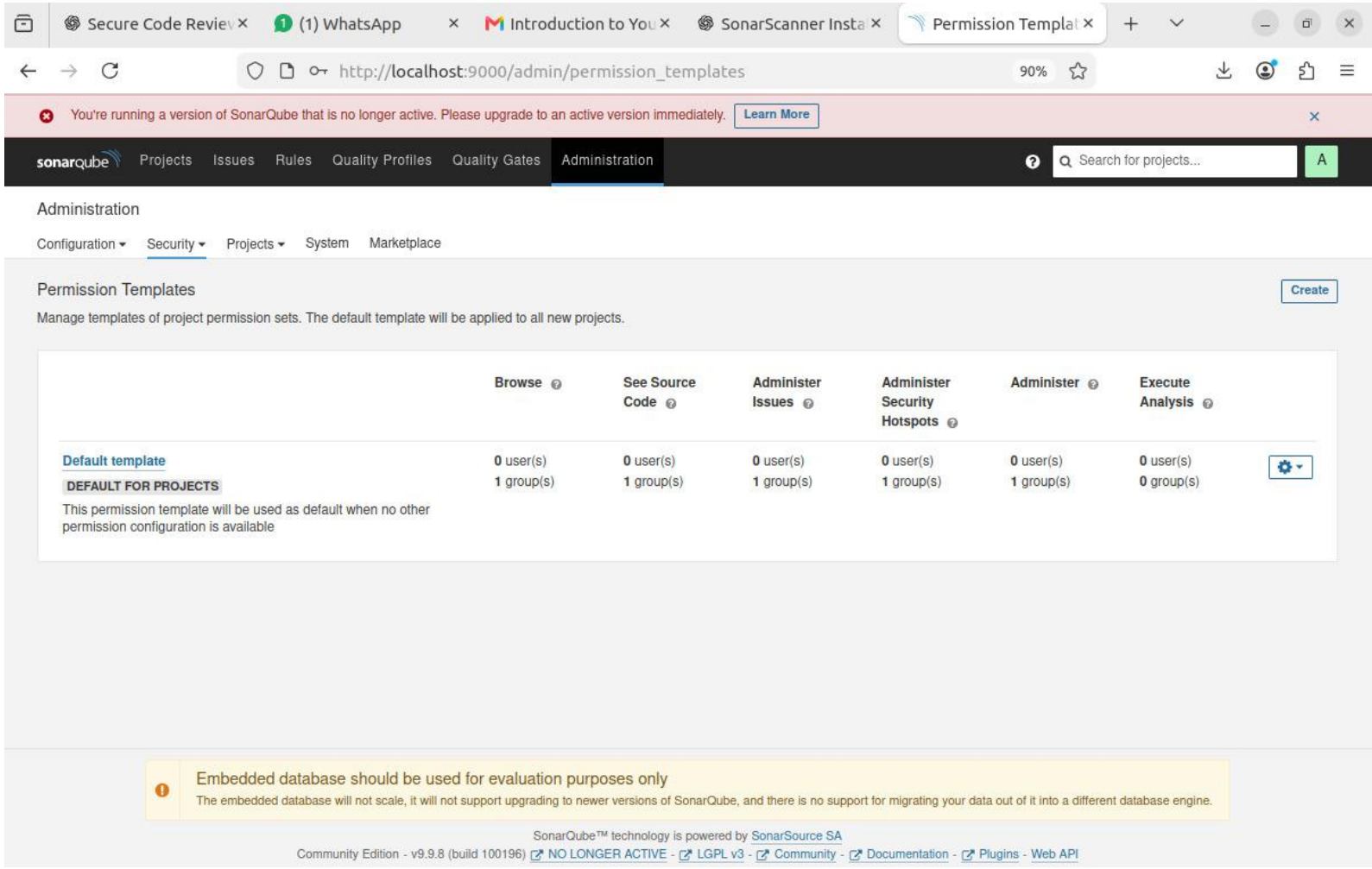
- **User Authentication**
- The Secure Code Review Framework provides secure user authentication through SonarQube's built-in login mechanism. Each user is assigned unique login credentials to access the platform.
- The authentication system provides:
- Secure user login.
- User identity verification.
- Session management.
- Controlled access to projects and analysis results.
- Only authenticated users can access the code review platform, thereby improving system security.



- **User Management**
- Multiple users were created within the system to support collaborative code review activities. Different users represent various roles involved in the software development process.
- The created users include:
- **Administrator**
- **Administrator User**
- **Reviewer User**
- Each user account contains separate login credentials and permissions.



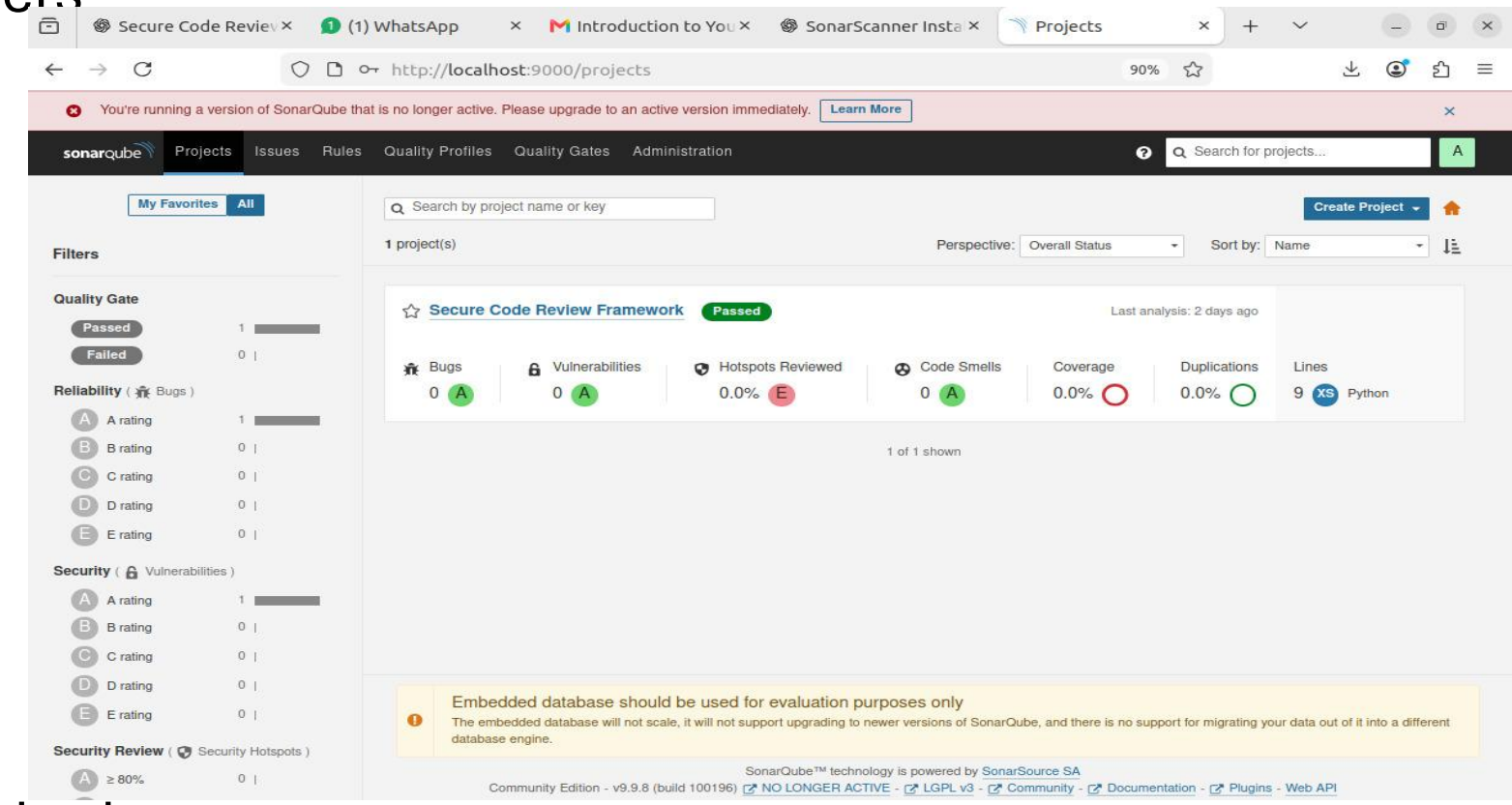
- **Permission Templates**
- Permission templates were configured to simplify access management across projects. The default project permission template includes several predefined privileges.
- The configured permissions include:
- Browse Projects
- See Source Code
- Administer Issues
- Administer Security Hotspots
- Execute Analysis
- Project Administration
- Permission templates help maintain consistent security policies across multiple projects.



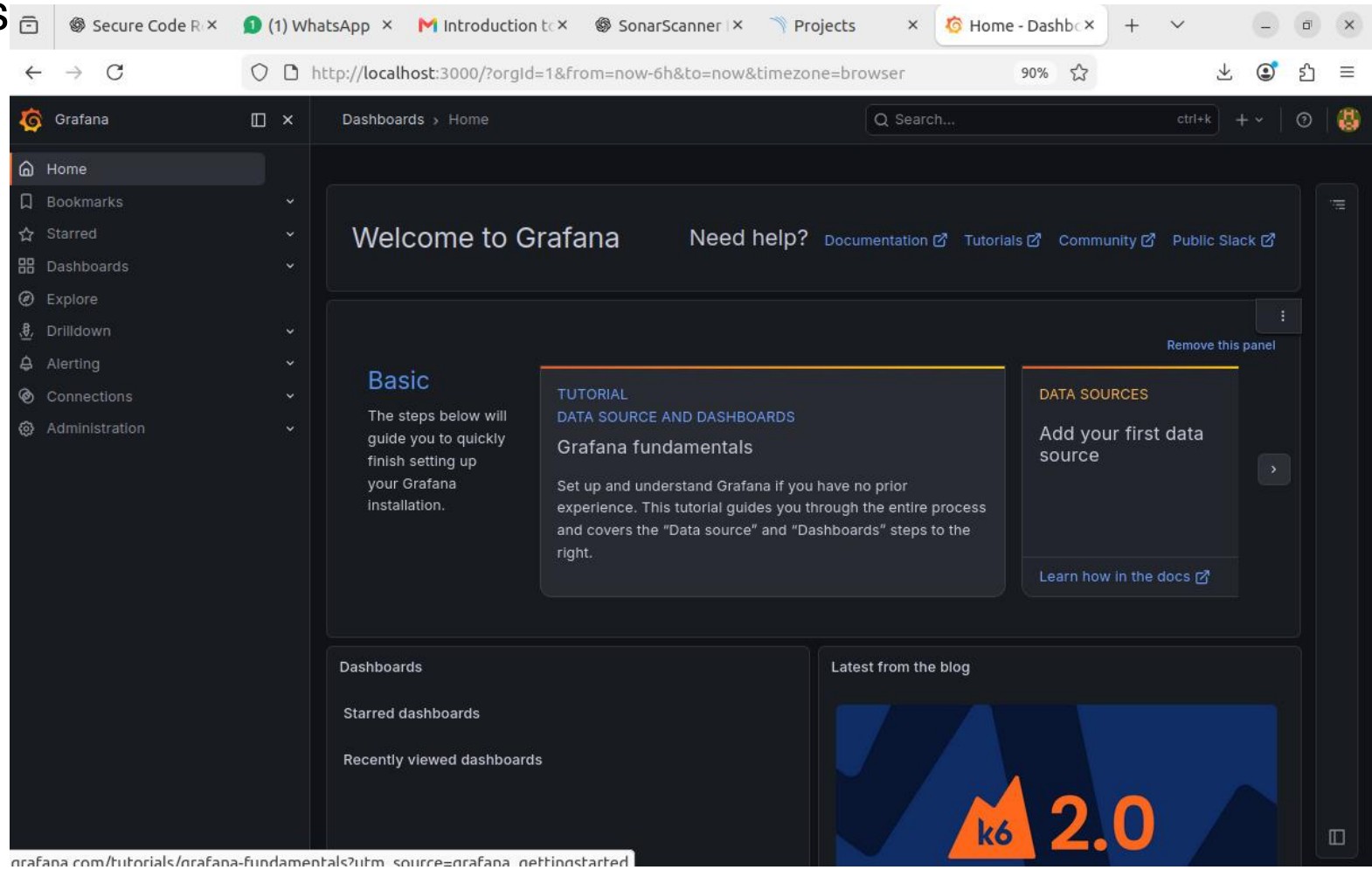
6. Reporting & Visualization

- **Introduction**
- Reporting and visualization play a crucial role in secure software development by presenting security findings, code quality metrics, and vulnerability information in an understandable manner. The Secure Code Review Framework uses SonarQube and Grafana dashboards to provide visual representations of analysis results, enabling developers and administrators to monitor project quality and security status effectively.

- **SonarQube Reporting Dashboard**
- SonarQube provides a centralized dashboard that displays various software quality metrics and security information. The dashboard summarizes the analysis results of the project and helps users identify potential issues.
- The dashboard includes:
- Quality Gate status.
- Bug count.
- Vulnerability count.
- Security Hotspots.
- Code Smells.
- Code Coverage.
- Code Duplication.
- Reliability and Security ratings.
- The project dashboard indicates whether the code satisfies the predefined quality standards.



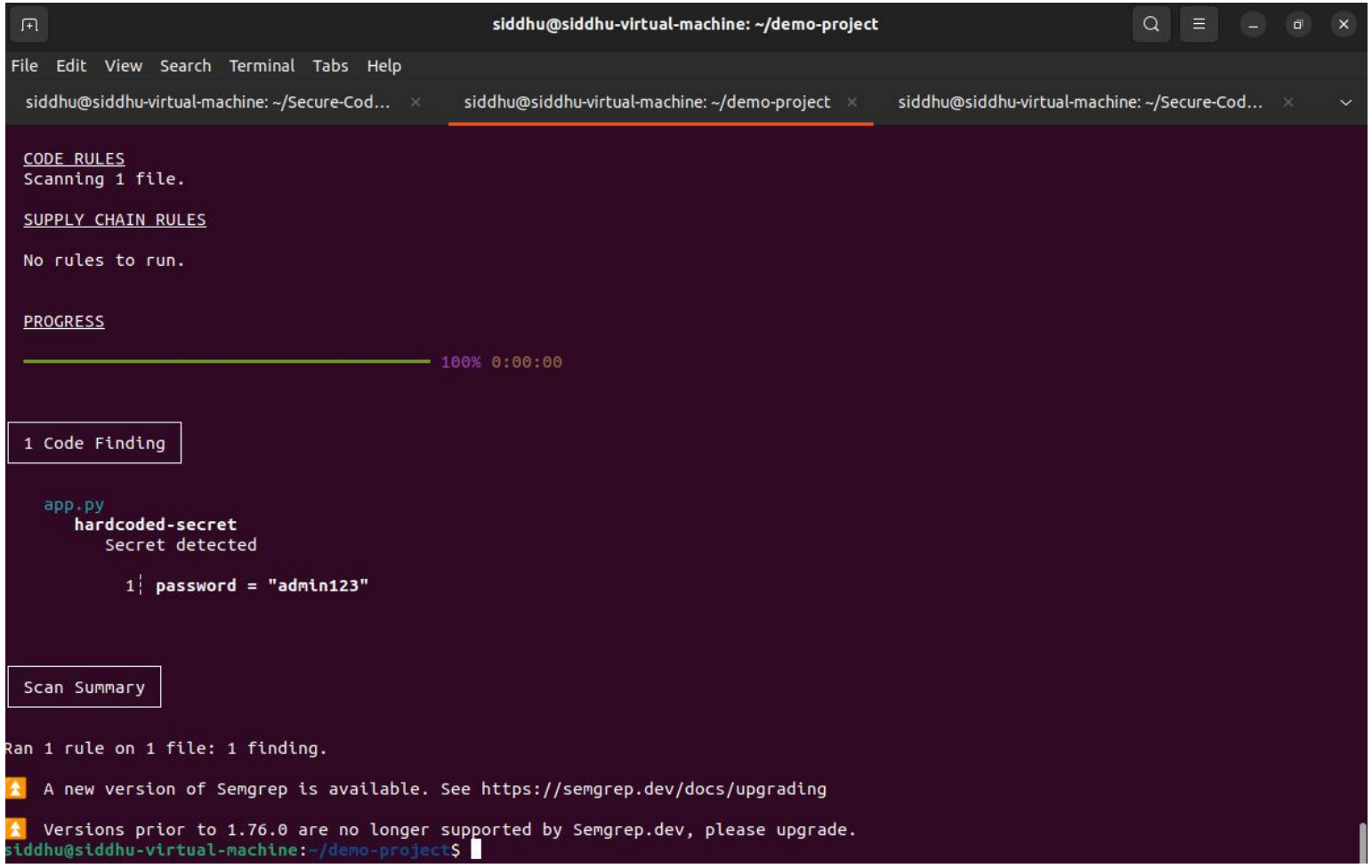
- **Grafana Visualization Dashboard**
- Grafana was configured to provide interactive dashboards and visual monitoring capabilities. It enables users to visualize system metrics, security information, and project performance through graphical representations.
- Grafana offers:
- Interactive dashboards.
- Real-time monitoring.
- Custom visualizations.
- Dashboard management.
- Data source integration.
- Performance analysis.
- The dashboard allows administrators to monitor project activities and system status efficiently.



- **Conclusion**
- The Reporting and Visualization module successfully integrates SonarQube and Grafana to provide comprehensive dashboards and analytical reports. The combination of detailed reporting, severity-based filtering, and visual monitoring improves the effectiveness of secure code reviews and assists organizations in maintaining high software quality standards.

7. Usability & Collaboration Features

- **Introduction**
- Usability and collaboration features improve the effectiveness of secure code review by enabling developers, reviewers, and administrators to work together efficiently. The Secure Code Review Framework supports customizable security rules and collaborative issue management to enhance code quality and maintain organizational security standards



Semgrep Custom Rule Detection of Hardcoded Secret

- The figure demonstrates the execution of the Semgrep security scanner. The custom security rule successfully detected a hardcoded password stored in the app.py file. This finding illustrates the capability of the framework to identify organization-specific security violations and assist developers in remediating vulnerabilities before deployment.

- **Conclusion**
- The Usability and Collaboration module improves the effectiveness of the Secure Code Review Framework by supporting custom security rules and collaborative vulnerability management. The integration of Semgrep-based rule detection enables organizations to enforce security policies while facilitating teamwork during code review activities.

8. Performance Optimization

- Performance optimization is important because security scanning should not slow down the software development process. In this project, SonarScanner is configured to scan only the required source files and avoid unnecessary files, reducing scan time and system load.
- **Objectives**
- Minimize the impact of security scans on build time.
- Improve scanner execution speed.
- Reduce CPU and memory usage.
- Generate reports quickly.
- Maintain accurate vulnerability detection.

- **Conclusion**
- The project **"Implementing a Secure Framework for a Code Review Tool"** successfully achieved the objective of developing a scalable and secure code review environment capable of identifying vulnerabilities, insecure coding practices, and software quality issues across multiple programming languages.
- The framework integrates industry-standard static analysis and vulnerability scanning tools such as SonarQube, Semgrep, and OWASP Dependency-Check to perform comprehensive security analysis. Security rules based on OWASP Top Ten, SANS CWE, and CERT secure coding standards were implemented to ensure adherence to recognized security guidelines and best practices.
- The system provides detailed remediation guidance, enabling developers to understand vulnerabilities and apply secure coding techniques effectively. Integration with version control systems allows automated scanning during code commits and pull requests, thereby supporting secure software development practices within the CI/CD pipeline.
- Authentication and Role-Based Access Control (RBAC) mechanisms were implemented to ensure secure access management for administrators, reviewers, and developers. Reporting dashboards and compliance reports improve visibility into security issues and support auditing and decision-making processes.
- Collaboration features such as issue tracking, commenting, and custom rule creation enhance team productivity and allow organizations to enforce their own security policies. Performance optimization techniques, including selective source scanning and exclusion of unnecessary files, minimize the impact of security analysis on build performance.
- Overall, the developed framework demonstrates that integrating security into the code review process significantly improves software quality, reduces vulnerabilities, and promotes secure development practices. The proposed solution provides a practical, extensible, and efficient approach for organizations seeking to implement secure code review processes within modern software development environments.
- **Future Scope**
- Integration with advanced AI-based vulnerability detection systems.
- Support for cloud-native and container security scanning.
- Automated security remediation suggestions using machine learning.
- Integration with DevSecOps pipelines for continuous security monitoring.
- Enhanced compliance reporting for industry standards such as ISO 27001, PCI-DSS, and GDPR.
- In conclusion, the project establishes a strong foundation for secure code review and contributes to the development of reliable, maintainable, and secure software systems.